Python: Automating an IP Allow-List Update

Revoking access to restricted content with a file-processing algorithm

Type	Python automation project
Tools	Python 3, Jupyter
Skills	File I/O, string and list methods, loops, conditionals, functions, input validation, testing
Source	Google Cybersecurity Professional Certificate, Course 7 (*Automate Cybersecurity Tasks with Python*), extended
Code	`allow_list_automation.ipynb` and `update_allow_list.py`

Project description

Access to restricted content is controlled by an allow list of IP addresses in `allow_list.txt`. A separate remove list holds addresses that should no longer have access. I wrote an algorithm that updates the file automatically, so access is revoked consistently and without manual mistakes.

1. Open and read the file

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
    ip_addresses = file.read()
```

The `with` statement opens the file in read mode (`"r"`) and closes it automatically when the block ends. `.read()` returns the whole file as a single string.

2. Convert the string into a list

```
ip_addresses = ip_addresses.split()
header, ip_addresses = ip_addresses[0], ip_addresses[1:]
```

`.split()` breaks the string on whitespace, so each IP address becomes a list element. The first element is the column header (`ip_address`), so I keep it separate from the addresses.

3. Iterate through the remove list and remove addresses

```
remove_list = ["192.168.97.225", "192.168.158.170",
               "192.168.201.40", "192.168.58.57"]

for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)
```

The `for` loop goes through each element of **`remove_list`**. The `if` check makes sure the address is in the allow list before calling `.remove()`. Without it, `.remove()` would raise an error for an address that isn't there.

Bug fixed: My first version looped over `ip_addresses` while removing items from that same list. Changing a list while you iterate over it makes Python skip elements. Looping over `remove_list` avoids this.

4. Write the updated list back to the file

```
with open(import_file, "w") as file:
    file.write("\n".join([header] + ip_addresses))
```

`"\n".join()` turns the list back into a string with one address per line. Opening the file in write mode (`"w"`) replaces its contents, so the removed addresses lose access straight away.

5. Extension: a reusable, validated function

```
import ipaddress

def update_allow_list(path, remove_list):
    for ip in remove_list:
        ipaddress.ip_address(ip)      # reject malformed addresses
    with open(path) as file:
        lines = file.read().split()
    header, current = lines[0], lines[1:]
    to_remove = set(remove_list)
    kept    = [ip for ip in current if ip not in to_remove]
    removed = [ip for ip in current if ip in to_remove]
    with open(path, "w") as file:
        file.write("\n".join([header] + kept))
    return removed                    # audit trail
```

In testing, the function removed all four addresses (17 → 13 entries), and automated checks confirmed that none of them were left in the file.

Summary

The algorithm reads the allow list, converts it to a list, removes every address on the remove list, and writes the result back to the file. I then made it production-ready: I fixed a modify-while-iterating bug, validated input addresses, returned removed addresses for auditing, and added tests. Automating access-control maintenance like this cuts down on human error and makes revocation fast and repeatable.